Prompt para o Claude Code - Trabalho Prático 02: Sistema Configurável de RNA (MLP)

Contexto

Quero que você desenvolva, do zero, uma aplicação em **Python** (salvo se eu pedir outra linguagem) que construa, configure, treine, salve, carregue e use **Redes Neurais Artificiais do tipo Multilayer Perceptron (MLP)**. A aplicação deve ser **genérica e configurável**: diferentes arquiteturas têm que ser criadas pela interface (linha de comando ou web), **sem alterar o código-fonte**.

Este é um trabalho da disciplina de Inteligência Artificial. Por isso, os algoritmos centrais precisam ser **implementados à mão**, não importados prontos.

Restrições (obrigatórias)

1. Propagação direta, cálculo do erro, retropropagação e atualização dos pesos devem ser implementados por nós mesmos.
2. **Proibido** usar TensorFlow, Keras, PyTorch, scikit-learn MLP ou qualquer biblioteca com rede neural pronta ou diferenciação automática para treinar.
3. Permitido: *numpy* (matrizes), *pandas* ou *csv* (leitura de CSV), *json* (serialização), bibliotecas de interface (argparse, rich, Flask, Streamlit, Tkinter etc.).
4. A arquitetura **não pode ser fixa no código**: o usuário cria as redes pela aplicação.
5. Todas as funcionalidades precisam rodar sem mexer no código-fonte.
6. Código com organização modular: entidades, algoritmos de treino, gerenciamento de datasets e persistência em módulos separados.

Requisitos de desenvolvimento

1.1 Entidade Neurônio

Guarda pesos sinápticos e viés (bias); calcula a soma ponderada **$z = w \cdot x + b$**; aplica a função de ativação e devolve a saída **$a = f(z)$**. Mantém os valores intermediários do treino: **z**, a **saída** e o **delta $\delta = \partial E / \partial z$**, calculado na retropropagação.

1.2 Entidade Camada

Agrupa os neurônios de uma camada. Tem quantidade configurável de neurônios e **função de ativação própria**. Recebe as entradas da camada anterior e disponibiliza as saídas para a seguinte.

1.3 Entidade Rede Neural

Representa a arquitetura completa. Permite definir: quantidade de entradas, número de camadas ocultas, neurônios por camada e neurônios de saída. Gerencia a propagação entre camadas e oferece as operações de treino e execução.

1.4 Funções de ativação

No mínimo: **Linear, Sigmoid, Tanh e ReLU**, cada uma com sua **derivada**. Camadas diferentes da mesma rede devem poder usar funções diferentes.

- Linear: f(z)=z, f'(z)=1
- Sigmoid: f(z)=1/(1+e^-z), f'(z)=f(z)(1-f(z))
- Tanh: f'(z)=1-f(z)^2
- ReLU: f(z)=max(0,z), f'(z)=1 se z>0, senão 0

1.5 Inicialização de pesos e vieses

- Pesos aleatórios com distribuição apropriada: **Xavier/Glorot** para Linear, Sigmoid e Tanh; **He** para ReLU.
- Permitir definir uma **semente (seed)** para reprodutibilidade.
- Vieses inicializados e armazenados junto com os pesos.

1.6 Propagação direta (Forward)

Processa a entrada por todas as camadas e produz o vetor de saída. Tem que funcionar para qualquer número de camadas e neurônios.

1.7 Erro: MSE

Erro Quadrático Médio entre saídas da rede e valores esperados. Deve calcular o MSE dos conjuntos de **treino, validação e teste**.

1.8 Retropropagação (Backpropagation)

- Usar a regra da cadeia para calcular os deltas da camada de saída e das ocultas.
- **Todos os deltas são calculados antes de qualquer atualização** de peso.
- Os deltas das camadas ocultas usam os **mesmos pesos da propagação direta** do exemplo (ou seja, calcule o delta com os pesos antes de atualizar).
- Atualizar pesos e vieses por **descida do gradiente**, com taxa de aprendizado configurável.

Fórmulas de referência (saída com MSE): $\delta_{saida} = (a - y) \cdot f'(z)$; $\delta_{oculta_j} = f'(z_j) \cdot \Sigma_k (w_{kj} \cdot \delta_k)$; gradiente do peso = $\delta \cdot$ entrada anterior; gradiente do viés = δ. Se usar $E = \frac{1}{2}\Sigma(y-a)^2$, o fator 2 some; documente a convenção escolhida no README.

1.9 Leitura e preparação de datasets

- Ler CSV e permitir **escolher as colunas de entrada e as colunas de saída** esperadas. Os dados já vêm limpos: não precisa tratar faltantes.
- Normalização dos atributos de entrada **opcional** (Min-Max ou Z-score).
- Verificar compatibilidade entre os dados e a arquitetura (nº de entradas e de saídas).
- Dividir em **treino, validação e teste** de forma **aleatória**, com percentuais configuráveis que somam 100%, e **semente** para reprodutibilidade.
- **Ordem obrigatória**: primeiro divide, depois normaliza. Os parâmetros (min e max, ou média e desvio-padrão) são calculados **só no treino** e aplicados aos três subconjuntos. As saídas

esperadas **não** são normalizadas.

1.10 Treinamento

- Usar **SGD puro**: os pesos e vieses são atualizados **após cada exemplo** de treino. É a única estratégia.
- Usar só o conjunto de treino para ajustar parâmetros.
- Ao fim de **cada época**, calcular e exibir o MSE de treino e o MSE de validação.
- Validação nunca atualiza pesos. Teste fica reservado para a avaliação final (calcule o MSE de teste só no fim).
- Sugestão: embaralhar o treino a cada época com a semente.

1.11 Persistência (salvar e carregar)

Salvar e carregar em **JSON** (ou outro formato documentado). O arquivo precisa conter:

- Quantidade de entradas.
- Quantidade de camadas e neurônios por camada.
- Função de ativação de cada camada.
- Pesos sinápticos de todos os neurônios.
- Vieses de todos os neurônios.
- Normalização usada (tipo: Min-Max ou Z-score, e seus parâmetros calculados no treino).

Ao carregar, a rede deve ser reconstruída e produzir **exatamente as mesmas saídas** para as mesmas entradas. Teste isso.

1.12 Previsão

- Executar a rede treinada sobre novos valores, **só com propagação direta**, sem alterar pesos nem vieses.
- Validar se a quantidade de valores informados bate com a quantidade de entradas.
- Se a rede foi treinada com normalização, aplicar a **mesma função e os mesmos parâmetros** salvos antes da propagação.

Interface

Pode ser linha de comando, web ou gráfica. Todas as funções acima devem ser acessíveis ao usuário, por exemplo: criar rede (informando camadas, neurônios e ativações), carregar dataset e escolher colunas, dividir dados, configurar treino (épocas, taxa de aprendizado, seed, normalização), treinar mostrando o MSE por época, testar, salvar, carregar e prever.

Organização sugerida do projeto

```
projeto/
  README.md            (como rodar, convenções, exemplos)
  requirements.txt
  main.py             (interface)
```

```
rna/
  __init__.py
  ativacoes.py      (Linear, Sigmoid, Tanh, ReLU + derivadas)
  neuronio.py       (entidade Neurônio)
  camada.py         (entidade Camada)
  rede.py           (entidade Rede Neural, forward)
  inicializacao.py  (Xavier, He, seed)
  erro.py           (MSE)
  treino.py         (backprop + SGD + épocas)
  dataset.py        (CSV, colunas, divisão, normalização)
  persistencia.py   (salvar/carregar JSON)
testes/
  test_*.py
dados/               (CSVs de exemplo)
```

Testes que você deve fazer

1. Verificar a derivada de cada ativação (comparar com diferença finita).
2. Verificar o gradiente da rede com gradiente numérico em um exemplo pequeno.
3. Treinar um problema simples (ex.: XOR ou AND/OR) e mostrar que o erro cai.
4. Salvar, carregar e comparar as saídas: têm que ser iguais.
5. Confirmar que a normalização usa só o treino (conferir os parâmetros salvos).
6. Mesma seed = mesmos pesos iniciais e mesma divisão.
7. Previsão com número errado de entradas deve dar erro claro.

Entrega

- Código-fonte completo do projeto (com README explicando como instalar, rodar e usar cada função).
- Um vídeo explicativo é feito pelo grupo (não pelo Claude): explicar a estrutura do código e as funções principais, descrever tecnologias e decisões, e demonstrar a execução com resultados.

O que eu quero de você agora

1. Primeiro, proponha a estrutura de arquivos e as assinaturas principais das classes, e espere meu ok.
2. Depois implemente módulo por módulo, na ordem: ativações, neurônio, camada, rede, erro, dataset, treino, persistência, previsão, interface.
3. Ao final, rode os testes e me mostre o resultado, junto com um exemplo de uso (comandos de terminal) para o vídeo.
4. Deixe claro no README quais suposições você fez (ex.: convenção do erro, formato do CSV, linguagem).